Red profesional sobre Neo4j

Guía del proyecto para el equipo

Qué es, cómo se levanta, cómo está hecho y dónde está cada cosa en el código.

Proyecto de Investigación y Desarrollo · Bases de Datos (202610-2497) · Universidad del Norte · 8 de noviembre de 2026

Cómo leer esta guía

- **Si tienes 10 minutos:** secciones 1, 2 y 3. Con eso entiendes qué es, cómo se prende y cómo fluye una petición.
- **Antes de cambiar código:** secciones 7 y 9. Dicen dónde está cada cosa y qué reglas no romper.
- **Para la demo:** sección 12 y el archivo `docs/guion-demo.md`.
- **Si algo falla:** sección 13.

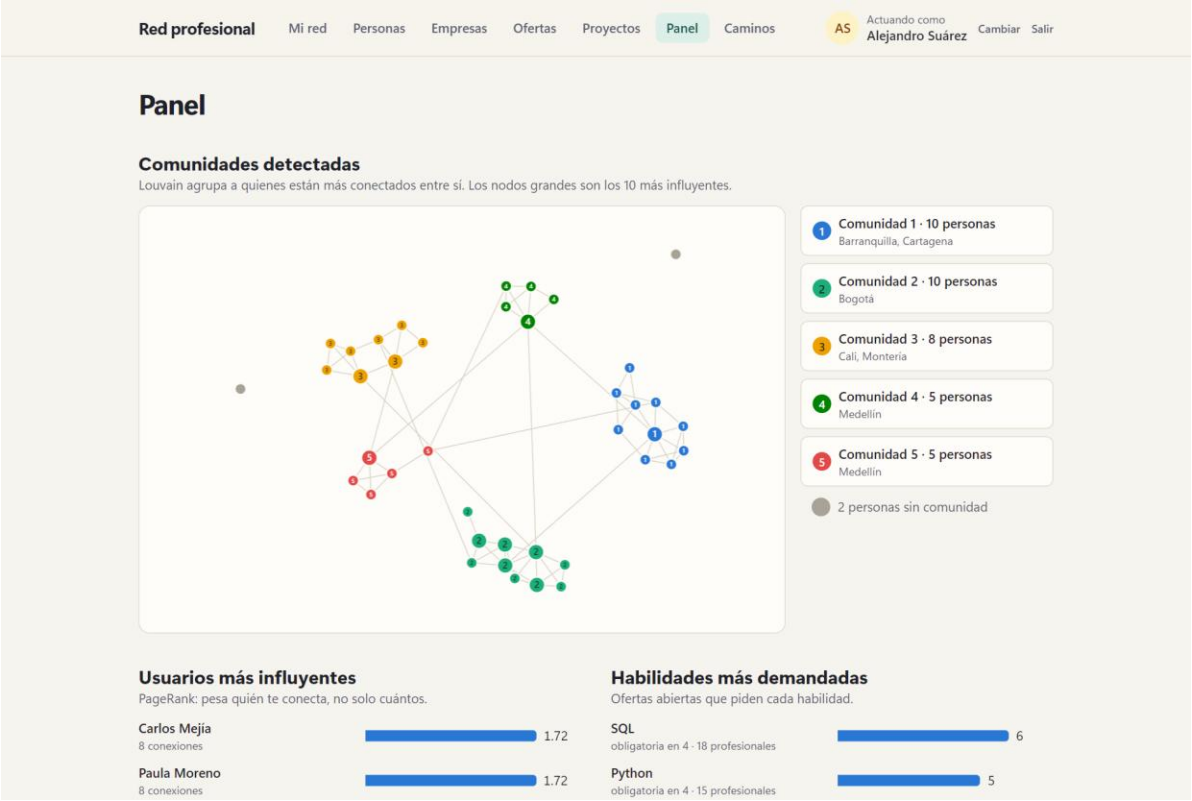
1. El proyecto en un minuto

Es una red profesional al estilo de LinkedIn, simplificada, construida sobre Neo4j, una base de datos de grafos. Las personas se conectan entre sí, trabajan en empresas, tienen habilidades, participan en proyectos y se postulan a ofertas. Sobre esa red la aplicación recomienda contactos, empleos, candidatos y colaboradores, encuentra caminos ("¿por quién llego a esta empresa?") y muestra analítica: comunidades e influencia.

El enunciado además pide responder con argumentos cuándo una base de grafos supera a una relacional y cuándo no. Eso lo respondimos midiendo: el benchmark compara Neo4j con PostgreSQL usando los mismos datos.

Pieza	Qué es	Dónde
Base de datos	Neo4j 5.26 con Graph Data Science, en Docker. Esquema, semilla de 40 personas y las 13 consultas del entregable	`db/`
Backend	API REST en Node.js y TypeScript (Express) que valida cada petición y la convierte en Cypher	`backend/`
Frontend	La aplicación web en React: 7 vistas, entre ellas el grafo dibujado	`frontend/`
Benchmark	Neo4j contra PostgreSQL con los mismos datos, hasta 100 000 personas	`benchmark/`
Documento de entrega	Teoría, modelo, scripts, consultas, aplicación, pruebas e investigación	`docs/Entrega - Red profesional sobre Neo4j.docx`
Demo	Guion de 15 minutos, script de preparación y ensayo automático	`docs/guion-demo.md`, `scripts/`

Dato	Valor
Modelo	6 tipos de nodo y 12 tipos de relación
Semilla	110 nodos y 554 relaciones: 40 personas, 8 empresas, 15 ofertas, 12 proyectos, 29 habilidades y 6 ciudades
API	56 endpoints en 10 grupos
Pruebas	337 del backend, 104 de extremo a extremo, 21 del benchmark y un ensayo automático de la demo
Pruebas de mutación	245 de 246 errores sembrados a propósito fueron detectados



El Panel: la red coloreada por las 5 comunidades que encuentra Louvain, con los más influyentes según PageRank.

2. Cómo levantarlo en tu máquina

Necesitas	Para qué
Docker Desktop	Corre Neo4j (y PostgreSQL, si vas a usar el benchmark)
Node.js 22.18 o superior	Backend, frontend y benchmark
Google Chrome	Las pruebas con Playwright usan el Chrome instalado
PowerShell (viene con Windows)	El script de preparación `scripts\preparar-demo.ps1`

1. Abre Docker Desktop y espera a que diga **Engine running**.
2. Instala las dependencias una sola vez: `npm install` dentro de `backend/` y dentro de `frontend/` (y en `benchmark/` si lo vas a usar).
3. Desde la raíz del repositorio, en PowerShell, corre el script de preparación. Enciende Neo4j, carga el esquema y la semilla, y comprueba que queden 110 nodos y 554 relaciones.
4. Enciende el backend: `npm run dev` dentro de `backend/`. Queda en http://localhost:3000 (prueba http://localhost:3000/salud).
5. Enciende el frontend: `npm run dev` dentro de `frontend/` y abre http://localhost:5173.

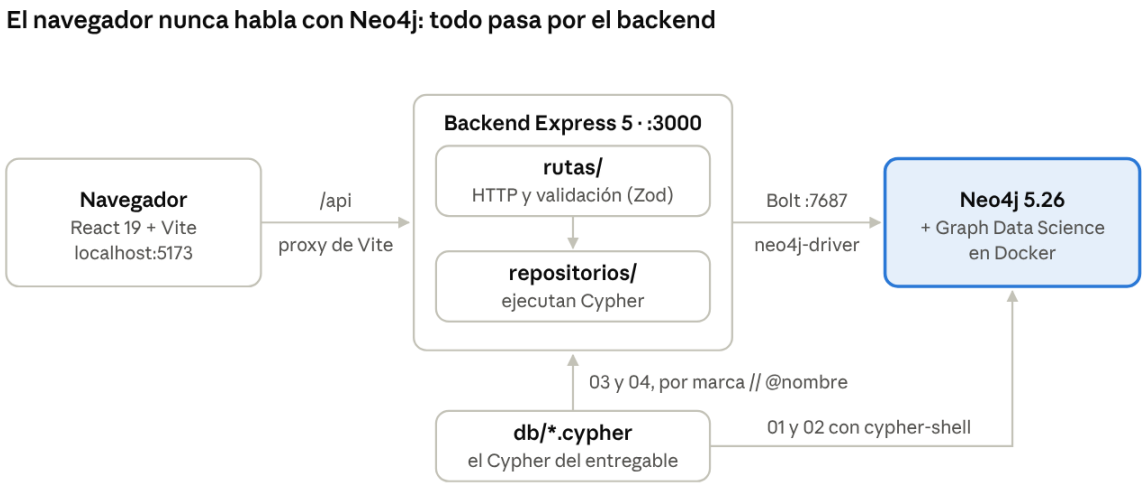
6. Arriba a la derecha, en **Actuar como**, elige a alguien (por ejemplo Alejandro Suárez). No hay inicio de sesión: eso es todo.

```
powershell -ExecutionPolicy Bypass -File scripts\preparar-demo.ps1
cd backend; npm run dev        # una terminal
cd frontend; npm run dev       # otra terminal
```

Puerto	Qué es
5173	La aplicación (Vite). Las llamadas a `/api` las reenvía al backend
3000	El backend (API REST)
7474	Neo4j Browser, para ver el grafo y correr Cypher a mano. Usuario `neo4j`, contraseña `redprofesional123`
7687	Bolt: el protocolo con el que el backend habla con Neo4j
7475 y 7688	Neo4j del benchmark (contenedor aparte, no toca la base de la demo)
5433	PostgreSQL del benchmark

> **Para recordar:** la contraseña se cambia copiando `.env.example` como `.env` en la raíz; el backend la lee de `backend/.env`. Para volver a dejar la base como recién instalada, corre otra vez `preparar-demo.ps1`.

3. Cómo está armado



Tres capas: el navegador nunca habla con Neo4j, todo pasa por el backend.

El navegador pide datos a `/api/...`. Vite, el servidor de desarrollo del frontend, reenvía esas peticiones al backend en el puerto 3000 (un **proxy**, por eso no hace falta configurar CORS). El backend valida lo que llega con Zod, ejecuta la consulta Cypher que corresponde con el driver oficial de Neo4j y devuelve JSON.

Una decisión central: **todo el Cypher vive en archivos `db/*.cypher`, cada consulta bajo una marc`//@nombre`. El backend carga esas consultas al arrancar. Así, el archivo que entregamos (`db/03_consultas.cypher`) es exactamente lo que corre la aplicación, y cada consulta también se puede probar suelta en Neo4j Browser o `cypher-shell`.

Una petición de punta a punta

Ejemplo: la tarjeta **Empleos para ti** de la pantalla Mi red.

1. `frontend/src/paginas/MiRed.tsx`: el componente `EmpleosSugeridos` llama al hook `useEmpleosSugeridos(yo)`.
2. `frontend/src/api/recomendaciones.ts`: ese hook es un `useQuery` de TanStack Query con la llave `['recomendaciones', 'empleos', id]`; llama a `pedir('/usuarios/:id/recomendaciones/empleos')`.
3. `frontend/src/api/cliente.ts`: `pedir()` hace el `fetch` a `/api/...` y convierte cualquier error en `ErrorApi`, con un mensaje listo para mostrar.
4. `frontend/vite.config.ts`: el proxy manda `/api` a `localhost:3000`.
5. `backend/src/app.ts` monta las rutas; `backend/src/rutas/recomendaciones.ts` valida el id y llama al repositorio.
6. `backend/src/repositorios/recomendaciones.ts` toma el Cypher con `consulta('recomendarEmpleos')` y lo ejecuta con `leer()`.
7. `backend/src/consultas.ts` leyó ese Cypher de `db/03_consultas.cypher` al arrancar; `backend/src/neo4j.ts` lo corre en Neo4j y convierte el resultado a JSON.
8. De vuelta en el navegador, TanStack Query guarda la respuesta en caché y la tarjeta pinta cada empleo con su porqué: qué cumples, qué te falta y a quién conoces en la empresa.

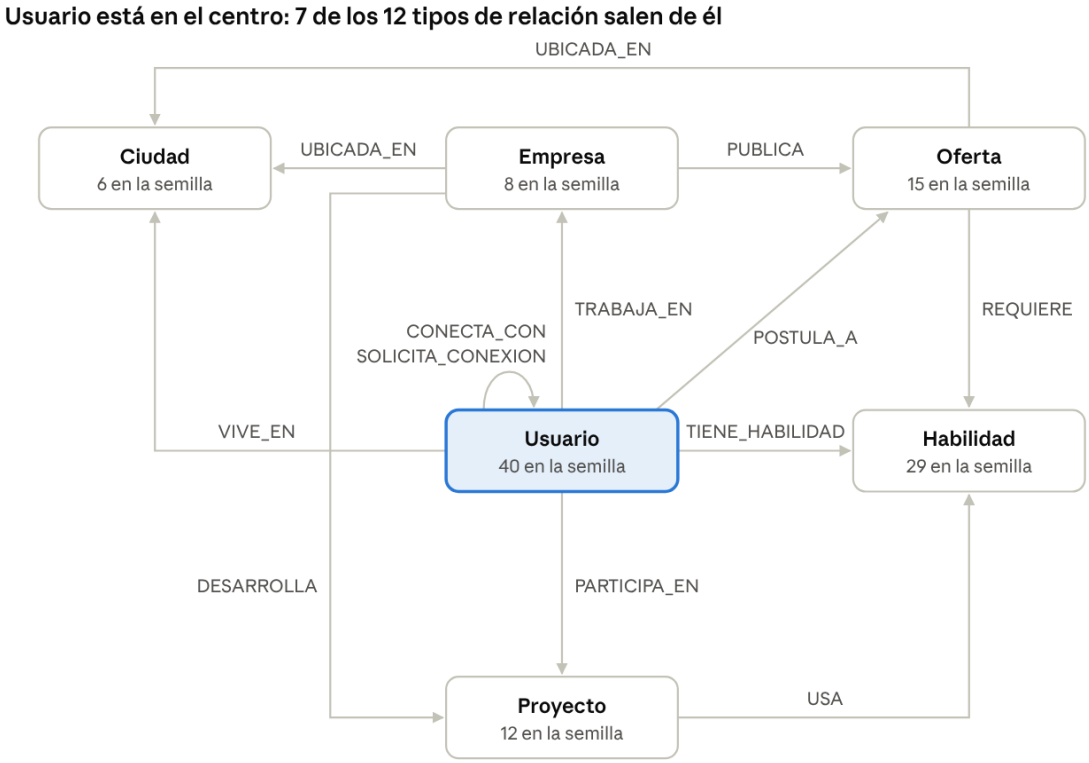
Una **escritura** (por ejemplo, Postularme) recorre el mismo camino, pero en el frontend usa `useEscritura` (`frontend/src/api/escritura.ts`): al terminar marca como vieja toda la caché, menos los catálogos, y cada pantalla vuelve a pedir lo que muestra. Las escrituras en Cypher están en `db/04_operaciones.cypher`.

4. Tecnologías

Tecnología	Para qué la usamos	Dónde
Neo4j 5.26 Community	Base de datos de grafos	`docker-compose.yml`, `db/`
Graph Data Science 2.13	PageRank y Louvain sobre una copia de la red en memoria	Consultas 11 a 13 de `db/03`
Cypher	Lenguaje de consultas de Neo4j	`db/*.cypher`

Tecnología	Para qué la usamos	Dónde
Docker Desktop	Contenedores de Neo4j y de PostgreSQL	`docker-compose.yml`
Node.js 22 y TypeScript	Lenguaje del backend, frontend y benchmark	Todo el código
Express 5	Servidor HTTP de la API	`backend/src/app.ts`, `rutas/`
Zod	Validar lo que llega, con mensajes en español	`backend/src/rutas/*`
neo4j-driver 6	Conexión oficial con Neo4j por Bolt	`backend/src/neo4j.ts`
Vitest y Supertest	Pruebas del backend contra Neo4j real	`backend/tests/`
React 19 y Vite	La aplicación web y su servidor de desarrollo	`frontend/src/`
React Router 7	Las rutas de cada pantalla	`frontend/src/main.tsx`
TanStack Query 5	Pedir datos al backend y guardarlos en caché	`frontend/src/api/`
Tailwind CSS 4	Estilos con clases en cada componente	`frontend/src/`
react-force-graph-2d	Dibujar el grafo en un canvas	`frontend/src/componentes/GrafoRed.tsx`
Playwright	Pruebas de extremo a extremo en Chrome	`frontend/e2e/`, `frontend/ensayo/`
PostgreSQL 17 y pg	La base relacional con la que comparamos	`benchmark/`
tsx	Correr TypeScript sin compilar (`npm run dev`)	`backend/`, `benchmark/`

5. El modelo de datos



Los 6 tipos de nodo y los 12 tipos de relación. Usuario está en el centro.

Nodo	Propiedades
`Usuario`	id, nombre, email, titular, aniosExperiencia, abiertoAOfertas, creadoEn
`Empresa`	id, nombre, sector
`Habilidad`	id, nombre, categoria
`Proyecto`	id, nombre, descripcion, estado
`Oferta`	id, titulo, nivel, modalidad, estado, publicadaEn
`Ciudad`	id, nombre, departamento

Las relaciones también guardan datos: `TRABAJA_EN {cargo, desde, actual}`, `TIENE_HABILIDAD {nivel}`, `PARTICIPA_EN {rol}`, `POSTULA_A {fecha, estado}`, `REQUIERE {nivelMinimo, obligatoria}`, `CONECTA_CON {desde}` y `SOLICITA_CONEXION {fecha}`.

Decisiones que conviene conocer

- **La conexión es mutua:** `CONECTA_CON` se guarda una sola vez y se consulta sin flecha, `(a)-[:CONECTA_CON]-(b)`.

- **La solicitud es otra relación:** `SOLICITA_CONEXION` vive mientras está pendiente; al aceptar se borra y se crea `CONECTA_CON`. Así las consultas y los algoritmos recorren solo conexiones reales, sin filtrar por un estado.
- **La ciudad es un nodo**, no un texto: se puede recorrer ("misma ciudad", comunidades por región).
- **Empleos anteriores** se guardan con `actual: false`: sirven para caminos y recomendaciones por excompañeros.
- **Ids:** los de la semilla son legibles (`usr-01`, `emp-03`, `of-06`); los que crea la aplicación son UUID.
- **Nombres:** etiquetas en PascalCase, relaciones en MAYÚSCULAS_CON_GUION_BAJO, propiedades en camelCase, todo en español.

6. Mapa del repositorio

```
red-profesional-grafos/
├── db/                  scripts Cypher: 00 limpiar, 01 esquema, 02 semilla, 03 consultas,
04 operaciones
├── backend/
│   ├── src/             app, conexión a Neo4j, cargador de consultas, errores
│   │   ├── rutas/       HTTP y validación (un archivo por recurso)
│   │   └── repositorios/  ejecutan el Cypher y aplican las reglas
│   └── tests/           337 pruebas contra Neo4j real
├── frontend/
│   ├── src/
│   │   ├── api/         hooks para pedir y escribir datos
│   │   ├── paginas/     una pantalla por archivo
│   │   ├── componentes/ piezas reutilizables, el grafo
│   │   └── contexto/    "Actuar como"
│   ├── e2e/             104 pruebas de extremo a extremo
│   └── ensayo/          la demo recorrida automáticamente
├── benchmark/           Neo4j contra PostgreSQL: generador, carga, medición, gráficas
├── scripts/             preparar-demo.ps1
├── docs/                enunciado, guion de la demo, capturas, documento de entrega
├── docker-compose.yml   contenedores (la demo y, con --profile benchmark, las bases
grandes)
├── README.md            cómo levantar todo, endpoints y comandos
└── CLAUDE.md            bitácora de decisiones y estado del proyecto
```

7. Lugares importantes del código

Base de datos (db/)

Archivo	Qué tiene	Cuándo tocarlo
`00_limpiar.cypher`	Borra todos los nodos y relaciones, por lotes	Casi nunca; lo usa el script de preparación
`01_esquema.cypher`	Restricciones de unicidad (como las llaves de SQL) e índices	Al agregar un tipo de nodo o una búsqueda nueva

Archivo	Qué tiene	Cuándo tocarlo
`02_datos_semilla.cypher`	La semilla: 40 personas en 4 regiones. Usa `MERGE`, así que se puede correr varias veces	Con cuidado: el guion de la demo y las pruebas dependen de estos datos
`03_consultas.cypher`	Las 13 consultas del entregable, cada una con un valor de ejemplo en `:param`	Al cambiar una recomendación, un camino o la analítica
`04_operaciones.cypher`	Todo lo demás: CRUD, vínculos, solicitudes, postulaciones, catálogos y el grafo para dibujar	Al agregar o cambiar una escritura

Backend (backend/src/)

Archivo	Qué hace
`server.ts`	Arranca la API y la apaga en orden (termina peticiones y cierra Neo4j)
`app.ts`	Arma Express y monta las rutas de cada recurso
`config.ts`	Lee `.env` y falla al arrancar si falta algo
`neo4j.ts`	El driver (uno solo, comparte conexiones). `leer`, `escribir`, `enTransaccion` y `entero()`. Reintentos limitados a 5 s
`consultas.ts`	Carga el Cypher de `db/` por su marca `// @nombre`. `consulta('nombre')` lo devuelve; un nombre mal escrito falla al arrancar
`errores.ts`	`ErrorApp` con su código HTTP (404, 409…), 503 si Neo4j no responde, y la forma de cada error para el cliente
`rutas/*.ts`	Un archivo por recurso: valida con Zod y llama al repositorio
`repositorios/*.ts`	Ejecutan el Cypher y aplican las reglas, por ejemplo que una empresa con ofertas no se borra (409)
`repositorios/analitica.ts`	Graph Data Science: recrea la proyección en cada llamada, de una en una (`enCola`)
`../tests/*.test.ts`	Pruebas contra Neo4j real. `tests/ayudas.ts` crea datos marcados y los limpia

Frontend (frontend/src/)

Archivo	Qué hace
`main.tsx`	Rutas de las pantallas (React Router) y la caché de TanStack Query (10 s de vigencia)

Archivo	Qué hace
`componentes/Marco.tsx`	El menú y el selector "Actuar como"
`contexto/UsuarioActual.tsx`	La persona que se está representando; se guarda en el navegador (`localStorage`)
`api/cliente.ts`	`pedir()`: todas las llamadas al backend pasan por aquí
`api/escritura.ts`	`useEscritura`: toda escritura refresca la caché al terminar
`api/*.ts`	Un archivo por recurso con sus hooks (`useOferta`, `useCandidatos`, `usePostularse`…)
`paginas/*.tsx`	Una pantalla por archivo: MiRed, Personas, Perfil, Empresas, Ofertas, Proyectos, Panel, Caminos
`componentes/GrafoRed.tsx`	El dibujo del grafo: fuerzas, colores, camino resaltado, clic en nodos. `GrafoDiferido.tsx` lo carga solo cuando hace falta
`colores.ts`	La paleta de comunidades, validada para que los colores se distingan
`utiles.ts`	`useSinRepetir` y `useAccion` (evitan el doble clic), `mesYAnio` (fechas), `listaLegible`
`componentes/FormularioSimple.tsx`, `Detalle.tsx`	Formularios y páginas de detalle de empresas, ofertas y proyectos
`../e2e/`, `../ensayo/`	Pruebas en Chrome. `e2e/ayudas.ts` cuenta píxeles para probar el canvas

Benchmark (benchmark/)

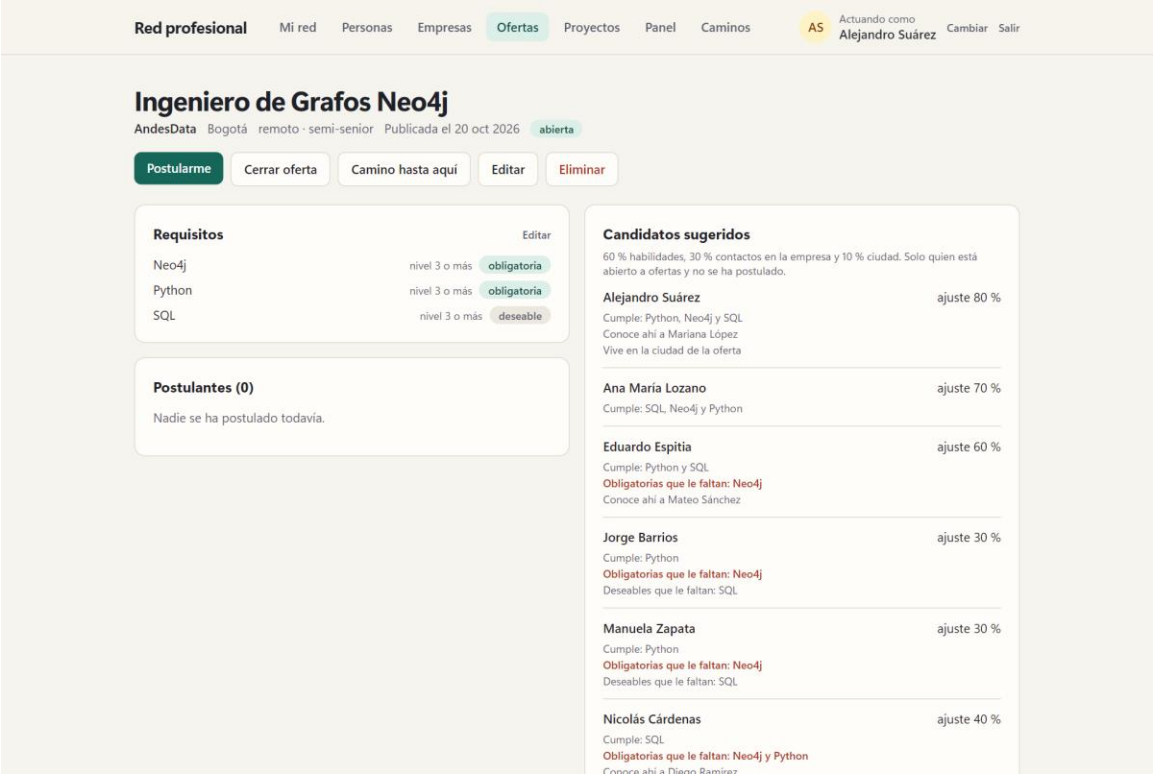
Archivo	Qué hace
`src/generar.ts`, `src/modelo.ts`	Genera la red sintética con semilla fija (comunidades por ciudad y personas muy conectadas)
`src/cargar.ts`	Carga los mismos CSV en Neo4j y PostgreSQL y verifica los conteos
`src/consultas.ts`	Las 14 consultas, cada una en Cypher y en SQL
`src/medir.ts`	Comprueba que las dos bases respondan igual y mide; guarda los planes de ejecución
`src/graficas.ts`	Genera las gráficas y tablas desde los resultados
`src/demo.ts`	El camino más corto en vivo para la demo (`npm run demo`)
`sql/`	Esquema relacional, restricciones y el procedimiento BFS de PostgreSQL
`RESULTADOS.md`	El informe con conclusiones y amenazas a la validez

8. Las 13 consultas

Todas están en `db/03_consultas.cypher`. Cada una tiene su endpoint y su lugar en la aplicación:

#	Qué responde	Endpoint	Dónde se ve
1	Usuarios con habilidades parecidas (Jaccard)	`GET /usuarios/:id/similares`	Perfil
2	Contactos en común entre dos personas	`GET /usuarios/:id/contactos-comunes/:otroId`	Perfil de otra persona
3	Ruta más corta de una persona a una empresa	`GET /usuarios/:id/ruta-empresa/:empresaId`	Caminos
4	Candidatos para una oferta, con su porqué	`GET /ofertas/:id/candidatos`	Detalle de oferta
5	Proyectos que comparten tecnologías o miembros	`GET /proyectos/relacionados`	Lista de proyectos
6	Habilidades más demandadas	`GET /analitica/habilidades`	Panel
7	Contactos recomendados	`GET /usuarios/:id/recomendaciones/contactos`	Mi red
8	Empleos recomendados	`GET /usuarios/:id/recomendaciones/empleos`	Mi red
9	Colaboradores para un proyecto	`GET /proyectos/:id/colaboradores`	Detalle de proyecto
10	Camino de una persona a una oferta	`GET /usuarios/:id/ruta-oferta/:ofertaId`	Caminos
11 y 12	Proyección en memoria y PageRank (influencia)	`GET /analitica/influyentes`	Panel
13	Comunidades (Louvain)	`GET /analitica/comunidades`	Panel

Las recomendaciones siempre devuelven **el porqué**. Candidatos y empleos usan el mismo puntaje: 60 % habilidades, 30 % contactos en la empresa (hasta 3) y 10 % ciudad, y ordenan primero por cuántas habilidades obligatorias faltan.



Detalle de una oferta: requisitos y candidatos sugeridos, cada uno con lo que cumple, lo que le falta y a quién conoce.

9. Reglas del proyecto (antes de cambiar algo)

Cada regla existe porque algo falló sin ella. Están todas en `CLAUDE.md`, con más detalle.

- **Cypher solo en `db/`**, con su marca `// @nombre`. Nada de Cypher dentro del código TypeScript.
- **Enteros hacia Neo4j con `entero()`:** un número de JavaScript llega como decimal, y `LIMIT` o las comparaciones fallan.
- **Fechas con `date({timezone: 'America/Bogota'})`**, nunca `date()`: el contenedor está en UTC y de noche daría el día siguiente. En el navegador, nunca `new Date('AAAA-MM-DD')`: usa `mesYAnio`.
- **Sin duplicados con `MERGE`**, no "revisar si existe y luego crear": entre las dos cosas cabe otra petición.
- **Listas siempre ordenadas:** `ORDER BY` antes de `collect()` y dentro de `COLLECT { }`; todo orden final desempata hasta el id. Sin eso, recargar la semilla cambia el orden de las respuestas.
- **Deduplicar por nodo** (`WITH DISTINCT c`), no por nombre: dos personas pueden llamarse igual.
- **Integridad a mano:** un grafo no tiene llaves foráneas. Empresa con ofertas no se borra (409); borrar una persona o una oferta se lleva sus postulaciones.
- **Validación solo en el backend** con Zod y mensajes en español; el frontend muestra el error junto a cada campo.
- **Escrituras con `useEscritura`** y botones con `useAccion` o `useSinRepetir`: `disabled` no frena un doble clic.

- **Datos de prueba marcados:** correos `@prueba.test` y nombres que empiezan con `[prueba]` ; la limpieza los busca así.
- **Correr también `npm run typecheck`:** Vitest no revisa tipos.

10. Pruebas

Qué	Comando	Cuántas	Necesita
Backend	`cd backend` y `npm test`	337	Neo4j con la semilla
Tipos	`npm run typecheck` en backend, frontend y benchmark	—	Nada
Extremo a extremo	`cd frontend` y `npm run e2e`	104	Neo4j con la semilla; enciende backend y Vite si faltan
Ensayo de la demo	`cd frontend` y `npm run ensayo`	1 (todo el guion)	Semilla recién cargada
Benchmark	`cd benchmark` y `npm test`	21	Nada

Además usamos **pruebas de mutación**: se mete a propósito un error pequeño en el código (quitar un filtro, invertir una comparación) y se comprueba que alguna prueba falle. Si ninguna falla, esa prueba no vigilaba nada. De 246 errores sembrados se detectaron 245; el que falta no cambia nada visible.

> **Ojo:** no corras las pruebas del backend y las de extremo a extremo al mismo tiempo: comparten la base y la limpieza de datos de prueba.

11. Benchmark: grafo contra relacional

Los mismos datos en Neo4j y en PostgreSQL, en redes de 1 000, 10 000 y 100 000 personas, con 14 consultas pareadas. PostgreSQL tuvo su mejor diseño (cada conexión en los dos sentidos e índices por ambos extremos). Antes de medir, cada consulta se comprueba: las dos bases deben responder igual y coincidir con un cálculo en memoria.

Consulta (100 000 personas)	Neo4j	PostgreSQL	Gana
Ida y vuelta sin trabajo	0,92 ms	0,31 ms	PostgreSQL 3×
Buscar por id	0,91 ms	0,31 ms	PostgreSQL 2,9×
Alcance a 4 saltos	18,9 ms	84,8 ms	Neo4j 4,5×
Camino más corto	1,16 ms	1 131 ms (CTE) / 282 ms (procedimiento)	Neo4j 975× / 243×
Habilidades más demandadas	6,87 ms	3,0 ms	PostgreSQL 2,3×
Candidatos para una oferta	403 ms	489 ms	Empate

Conclusión: el grafo gana cuando la consulta recorre la red a varios saltos, y su tiempo casi no cambia con el tamaño. El relacional gana en búsquedas cortas, agregaciones y carga masiva; en las consultas cortas casi toda su ventaja es el costo fijo de hablar con cada base. El informe completo está en `benchmark/RESULTADOS.md`.

```
docker compose --profile benchmark up -d
cd benchmark
npm run generar -- 100000; npm run cargar -- 100000
npm run medir -- 100000     # unos 2 minutos
npm run demo                # el camino más corto en vivo
```

12. La demo

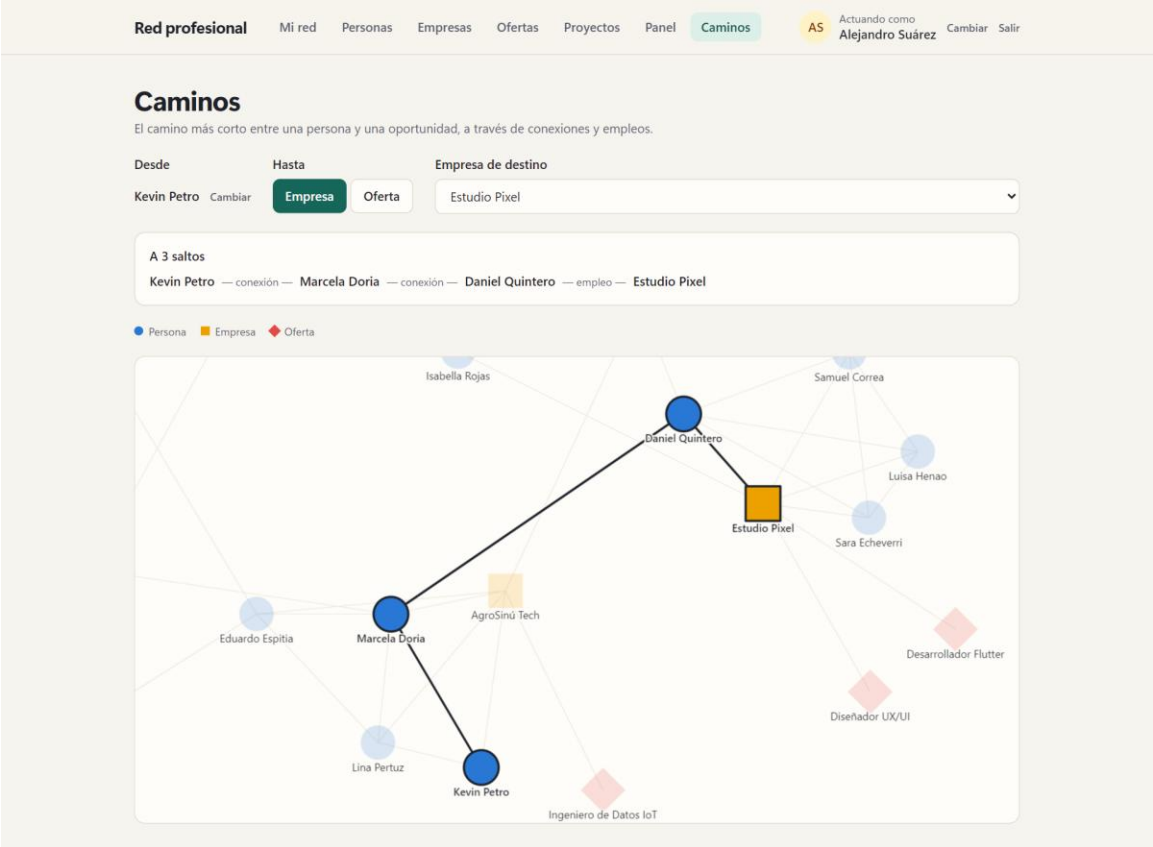
El guion completo, con qué decir, qué clic dar y qué debe verse en pantalla, está en `docs/guion-demo.md`. Dura unos 13 minutos y deja 2 de margen.

Reloj	Bloque	Qué se muestra
0:00	A. Problema y modelo	El diagrama del modelo y por qué un grafo
1:00	B. La base por dentro	Neo4j Browser: el esquema y los contactos en común dibujados
2:30	C. Una persona nueva	Se registra a Lucía Fernández y se la sigue: habilidades, empleo, conexión, oferta nueva, candidatos, postulación y borrado
7:00	D. Perfil, caminos y proyectos	Habilidades parecidas, contactos en común, Kevin Petro → Estudio Pixel en 3 saltos, colaboradores
8:45	E. Panel	Comunidades, PageRank y habilidades demandadas
10:00	F. Investigación	Las gráficas del benchmark y, si se quiere, el camino en vivo con 100 000 personas
12:30	G. Cierre	Dónde gana cada uno
13:30	H. Preguntas	Respuestas cortas preparadas en el guion

La víspera, corran el ensayo completo; si pasa, la demo funciona con los datos del guion:

```
powershell -ExecutionPolicy Bypass -File scripts\preparar-demo.ps1 -Ensayo -Benchmark
```

30 minutos antes, el mismo script sin `-Ensayo` deja la base limpia y calienta el benchmark. Después de la demo, córranlo otra vez: la demo deja creada una oferta.



Caminos: Kevin Petro llega a Estudio Pixel en 3 saltos, resaltado sobre la red.

13. Problemas comunes

Síntoma	Qué hacer
La app dice que no hay conexión con la base, o la API responde 503	Neo4j está apagado: `docker compose up -d` (o el script de preparación)
Docker Desktop muestra un error y se cierra; el log menciona `dockerInference`	Quedó un archivo viejo de un apagado brusco. Con Docker cerrado, renombra la carpeta `%LOCALAPPDATA%\Docker\run` (por ejemplo a `run.viejo`) y vuelve a abrir Docker
`cypher-shell` dice "No such file or directory" con una ruta `C:/Program Files/Git/scripts/...`	Git Bash cambió la ruta. Usa PowerShell, o antepón `MSYS_NO_PATHCONV=1` al comando
"Crear persona" da email repetido, o los datos no coinciden con el guion	Quedaron datos de un ensayo: corre `scripts\preparar-demo.ps1`
El ensayo dice "La base no está como la deja la semilla"	Lo mismo: `scripts\preparar-demo.ps1`
PowerShell no deja correr el script	Llámalo con `powershell -ExecutionPolicy Bypass -File scripts\preparar-demo.ps1`

Síntoma	Qué hacer
`npm run dev` dice que el puerto está en uso	Ya hay otra instancia encendida: ciérrala o usa esa
Pruebas que fallan de forma rara	No corras las del backend y las de extremo a extremo a la vez

14. Glosario

Término	Qué significa aquí
Grafo de propiedades	Modelo de Neo4j: nodos con etiquetas y relaciones con tipo, ambos con propiedades
Cypher	El lenguaje de consultas de Neo4j; se escribe como dibujos: `(a)-[:CONECTA_CON]-(b)`
Bolt	El protocolo con el que el driver habla con Neo4j (puerto 7687)
Adyacencia sin índice	Cada nodo apunta directo a sus relaciones: pasar a un vecino cuesta lo mismo con mil o un millón de nodos
Graph Data Science (GDS)	Librería de algoritmos de Neo4j; trabaja sobre una copia de la red en memoria (proyección)
PageRank	Influencia: pesa quién te conecta, no solo cuántos
Louvain	Detecta comunidades maximizando la modularidad, sin decirle cuántas buscar
Jaccard	Similitud: habilidades en común sobre todas las de los dos
Cierre triádico	Si A conoce a B y B conoce a C, A y C probablemente se conecten: base de las sugerencias de contactos
`allShortestPaths`	Todos los caminos más cortos; elegimos el primero por ids para que la respuesta no cambie
CTE recursiva	La forma de recorrer en SQL (`WITH RECURSIVE`); no puede detenerse al encontrar el destino
Semilla	Los datos de ejemplo que se cargan con `02_datos_semilla.cypher`; idempotente: correrla dos veces no duplica
Proxy de Vite	Reenvía `/api` del navegador al backend, así no hace falta CORS
Caché de TanStack Query	Guarda respuestas del backend; una escritura la marca como vieja para que se vuelva a pedir
Extremo a extremo (E2E)	Pruebas que usan la app como una persona, en el navegador real

Término	Qué significa aquí
Pruebas de mutación	Meter errores a propósito para comprobar que las pruebas los detectan